



Project Portfolio: Machine Learning for Insurance



About the Project

This project simulates real-world machine learning applications for a fictional insurance company called **Protect Your Tomorrow**. The goal is to solve common business problems using supervised machine learning techniques, including classification, regression, and sensitive data protection.



Main Objectives

1. Find **similar customers** based on their characteristics.
2. Predict whether a customer will **receive insurance payments** (binary classification).
3. Estimate **how many payments** they are likely to receive (regression).
4. Apply **data obfuscation** to protect personal information without degrading model performance.

This project could easily be extended to a real insurance company for:

- Personalized marketing (targeting similar customers)
 - Risk scoring for claim approval
 - Forecasting claim frequency
 - Privacy-preserving analytics in compliance with GDPR/LGPD
-



Data Exploration and Cleaning

```
# Loading and preparing data
df = pd.read_csv('/datasets/insurance_us.csv')
df = df.rename(columns={'Gender': 'gender',
                        'Age': 'age',
                        'Salary': 'income', 'Family members':
                        'family_members', 'Insurance benefits':
                        'insurance_benefits'})
df['age'] = df['age'].astype(int)
df['insurance_benefits_received'] = df['insurance_benefits'].apply(lambda x: 1
```

- The dataset contained 5,000 records and 5 columns: gender, age, income, family_members, insurance_benefits.
- No missing values.

- Features were type-corrected and transformed as needed.
-



Task 1: Similar Customers (kNN)

```
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import MaxAbsScaler

feature_names = ['gender', 'age', 'income', 'family_members']

def get_knn(df, n, k, metric):
    nbrs = NearestNeighbors(n_neighbors=k, metric=metric)
    nbrs.fit(df[feature_names])
    distances, indices = nbrs.kneighbors([df.iloc[n][feature_names]])
    return df.iloc[indices[0]].assign(distance=distances[0])
```

- Applied the **k-Nearest Neighbors** algorithm using Euclidean and Manhattan distances.
- Evaluated using **raw** and **scaled** data (MaxAbsScaler).

Conclusion:

- kNN is highly sensitive to data scale.
 - Scaling features ensures fairer and more balanced results across variables of different magnitudes.
-



Task 2: Classification (Insurance Payment Prediction)

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import f1_score, confusion_matrix

for k in range(1, 11):
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    print(f"k={k}, F1={f1_score(y_test, y_pred):.2f}")
```

Results:

- Dummy: F1 ranged from 0.00 to 0.20.
- kNN without scaling: Max F1 = 0.65.
- kNN with scaling: F1 up to **0.95**, demonstrating excellent performance with proper preprocessing.



Task 3: Linear Regression (Number of Payments)

```
class MyLinearRegression:
    def __init__(self):
        self.weights = None

    def fit(self, X, y):
        X2 = np.append(np.ones([len(X), 1]), X, axis=1)
        self.weights = np.linalg.inv(X2.T @ X2) @ X2.T @ y

    def predict(self, X):
        X2 = np.append(np.ones([len(X), 1]), X, axis=1)
        return X2 @ self.weights
```

Results:

- Raw data: RMSE = 0.34 | $R^2 = 0.43$
- Scaled data: RMSE = 0.34 | $R^2 = 0.66$

Conclusion:

- Scaling improves model fit and coefficient interpretation.



Task 4: Data Obfuscation

```
# Obfuscation
P = np.random.default_rng(seed=42).random((X.shape[1], X.shape[1]))
while np.linalg.det(P) == 0:
    P = np.random.default_rng().random((X.shape[1], X.shape[1]))
X_obf = X @ P
X_recovered = X_obf @ np.linalg.inv(P)
```

- Data was obfuscated by multiplying features by an **invertible random matrix (P)**.
 - The process made the data unreadable but reversible with high precision via P^{-1} .
 - Small differences in recovery are due to floating-point precision.
-



Task 5: Linear Regression with Obfuscated Data

```
# Regression using obfuscated data
model_obf = MyLinearRegression()
model_obf.fit(X_train @ P, y_train)
y_pred_obf = model_obf.predict(X_test @ P)
```

Identical results:

- $RMSE = 0.3436$
- $R^2 = 0.4305$

Conclusion:

- Obfuscation does not compromise model performance.
- Sensitive data can be protected without sacrificing predictive power.



General Conclusion

- **Preprocessing quality** was crucial to the success of the models.
- Simple models (like kNN and Linear Regression) perform very well when data is properly prepared.
- **Obfuscation with invertible matrices** is an effective technique for protecting sensitive data without degrading model performance.
- The project demonstrates a complete pipeline: from exploratory analysis to data security, applicable to real-world scenarios in companies handling confidential information.

If you'd like to see the code or detailed results, feel free to reach out!